

Setup & Deployment Guide - ResidenceCare

Phiên bản: 1.0.0

Ngày tạo: Tháng 5 năm 2026

Tác giả: Manus AI

1. Yêu Cầu Hệ Thống

1.1 Development Environment

Yêu Cầu	Phiên Bản	Ghi Chú
Node.js	18.0.0+	Khuyến nghị 20+
npm/pnpm	9.0.0+	Sử dụng pnpm 10.4.1
MySQL	8.0.0+	Hoặc TiDB
Git	2.30.0+	Để version control
Docker	20.10+	Optional, cho containerization

1.2 Production Environment

- OS:** Linux (Ubuntu 20.04+ khuyến nghị)
- CPU:** 2 cores minimum
- RAM:** 2GB minimum (4GB khuyến nghị)
- Disk:** 10GB minimum
- Network:** Kết nối internet ổn định

2. Local Development Setup

2.1 Clone Repository

```
# Clone project
git clone https://github.com/your-org/residence-care.git
cd residence-care

# Hoặc nếu đã có project
cd /home/ubuntu/residence-care
```

2.2 Cài Đặt Dependencies

```
# Cài đặt pnpm (nếu chưa có)
npm install -g pnpm@10.4.1

# Cài đặt dependencies
pnpm install

# Nếu gặp lỗi, xóa lock file và cài lại
rm pnpm-lock.yaml
pnpm install
```

2.3 Cấu Hình Environment Variables

Tạo file `.env.local` trong thư mục root:

```
# Database
DATABASE_URL=mysql://user:password@localhost:3306/residence_care

# Authentication
JWT_SECRET=your-secret-key-here-min-32-chars-long
VITE_APP_ID=your-manus-app-id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://manus.im/login

# Owner Info
OWNER_OPEN_ID=your-owner-open-id
OWNER_NAME=Your Name

# Manus APIs
BUILT_IN_FORGE_API_URL=https://api.manus.im/forge
BUILT_IN_FORGE_API_KEY=your-forge-api-key
VITE_FRONTEND_FORGE_API_URL=https://api.manus.im/forge
VITE_FRONTEND_FORGE_API_KEY=your-frontend-forge-key

# Analytics (Optional)
VITE_ANALYTICS_ENDPOINT=https://analytics.manus.im
VITE_ANALYTICS_WEBSITE_ID=your-website-id
```

2.4 Cấu Hình Database

```
# Tạo database mới
mysql -u root -p -e "CREATE DATABASE residence_care CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;"

# Hoặc sử dụng MySQL GUI tool (DBeaver, MySQL Workbench)
```

2.5 Chạy Database Migrations

```
# Generate migrations (nếu schema thay đổi)
pnpm drizzle-kit generate

# Xem generated SQL
cat drizzle/migrations/0001_*.sql

# Apply migrations (sử dụng Manus UI hoặc CLI)
webdev_execute_sql < drizzle/migrations/0001_*.sql
```

2.6 Seed Database (Optional)

```
# Tạo dữ liệu mẫu
node scripts/seed-db.mjs
```

2.7 Chạy Development Server

```
# Terminal 1: Backend
pnpm dev

# Hoặc chạy riêng
NODE_ENV=development tsx watch server/_core/index.ts

# Terminal 2: Frontend (nếu cần)
# Vite dev server chạy tự động với backend
```

Truy cập: <http://localhost:3000>

3. Development Workflow

3.1 Cấu Trúc Thư Mục

```
residence-care/
├─ client/           # Frontend React
├─ server/           # Backend Express + tRPC
├─ drizzle/          # Database schema
├─ scripts/          # Utility scripts
├─ docs/             # Documentation
├─ package.json      # Dependencies
└─ tsconfig.json     # TypeScript config
```

3.2 Thêm Tính Năng Mới

Step 1: Update Database Schema

```
// drizzle/schema.ts
export const newTable = mysqlTable("newTable", {
  id: int("id").autoincrement().primaryKey(),
  // ... columns
});
```

Step 2: Generate Migration

```
pnpm drizzle-kit generate
```

Step 3: Apply Migration

```
webdev_execute_sql < drizzle/migrations/0001_*.sql
```

Step 4: Add Query Helpers

```
// server/db.ts
export async function getNewTableData() {
  const db = await getDb();
  return db.select().from(newTable);
}
```

Step 5: Create tRPC Procedure

```
// server/routers/newFeature.ts
export const newFeatureRouter = router({
  list: protectedProcedure.query(({ ctx }) =>
    db.getNewTableData()
  ),
});
```

Step 6: Build Frontend

```
// client/src/pages/NewFeature.tsx
const { data } = trpc.newFeature.list.useQuery();
```

Step 7: Test

```
pnpm test
```

3.3 Testing

```
# Chạy tất cả tests
pnpm test

# Chạy test file cụ thể
pnpm test server/routers.test.ts

# Watch mode
pnpm test --watch

# Coverage
pnpm test --coverage
```

3.4 Code Quality

```
# Type checking
pnpm check

# Format code
pnpm format

# Lint (nếu có)
pnpm lint
```

4. Build Production

4.1 Build Frontend & Backend

```
# Build cả frontend và backend
pnpm build

# Output:
# - client/dist/      (Frontend bundle)
# - dist/             (Backend bundle)
```

4.2 Verify Build

```
# Check bundle size
ls -lh client/dist/
ls -lh dist/

# Test production build locally
NODE_ENV=production node dist/index.js
```

4.3 Build Artifacts

File	Kích Thước	Mô Tả
client/dist/index.html	10KB	HTML entry point
client/dist/assets/*.js	300KB	JavaScript bundles
client/dist/assets/*.css	50KB	Stylesheets
dist/index.js	2MB	Backend bundle

5. Deployment

5.1 Deploy trên Manus Platform (Recommended)

Advantages:

- Tự động scaling
- Built-in SSL/TLS
- Custom domain support
- Automatic backups
- One-click deployment

Steps:


```
# 1. Tạo checkpoint
webdev_save_checkpoint

# 2. Click "Publish" button trong Manus UI
# 3. Chọn domain (xxx.manus.space hoặc custom domain)
# 4. Chờ deployment hoàn tất
```

5.2 Deploy trên Railway/Render

Railway:

```
# 1. Tạo tài khoản Railway
# 2. Connect GitHub repository
# 3. Set environment variables
# 4. Deploy tự động

# Environment variables:
# DATABASE_URL
# JWT_SECRET
# VITE_APP_ID
# ... (xem .env.local)
```

Render:

```
# Tương tự Railway
# Hỗ trợ native Node.js environment
```

5.3 Deploy trên VPS (Self-hosted)

Prerequisites:

- Ubuntu 20.04+ server
- SSH access
- Domain name

Steps:

```
# 1. SSH vào server
ssh user@your-server.com

# 2. Cài đặt Node.js
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# 3. Cài đặt pnpm
npm install -g pnpm

# 4. Clone repository
git clone https://github.com/your-org/residence-care.git
cd residence-care

# 5. Cài dependencies
pnpm install --prod

# 6. Cấu hình environment
nano .env

# 7. Build
pnpm build

# 8. Chạy với PM2 (process manager)
npm install -g pm2
pm2 start dist/index.js --name "residence-care"
pm2 save
pm2 startup

# 9. Setup Nginx reverse proxy
sudo apt-get install nginx
# Cấu hình /etc/nginx/sites-available/residence-care
# Proxy requests đến localhost:3000
```

Nginx Configuration:

```
server {
    listen 80;
    server_name your-domain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

5.4 Deploy với Docker

Dockerfile:

```
FROM node:20-alpine

WORKDIR /app

# Copy files
COPY package.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --prod

COPY . .

# Build
RUN pnpm build

# Expose port
EXPOSE 3000

# Start
CMD ["node", "dist/index.js"]
```

Build & Run:

```
# Build image
docker build -t residence-care:latest .

# Run container
docker run -d \
  -p 3000:3000 \
  -e DATABASE_URL="mysql://..." \
  -e JWT_SECRET="..." \
  --name residence-care \
  residence-care:latest

# View logs
docker logs -f residence-care
```

6. Post-Deployment

6.1 Verify Deployment

```
# Check health
curl https://your-domain.com/health

# Check API
curl https://your-domain.com/api/trpc/auth.me

# Check logs
tail -f /var/log/residence-care.log
```

6.2 Setup SSL/TLS

Manus Platform: Automatic

Self-hosted with Let's Encrypt:

```
sudo apt-get install certbot python3-certbot-nginx
sudo certbot --nginx -d your-domain.com
```

6.3 Setup Monitoring

```
# PM2 Monitoring
pm2 install pm2-auto-pull
pm2 install pm2-logrotate

# System monitoring
sudo apt-get install htop
htop
```

6.4 Setup Backups

```
# Database backup script
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M%S)
mysqldump -u user -p database > backup_${DATE}.sql
gzip backup_${DATE}.sql

# Schedule with cron
# 0 2 * * * /path/to/backup.sh
```

7. Troubleshooting

7.1 Common Issues

Issue	Giải Pháp
Port 3000 already in use	<code>lsof -i :3000</code> và kill process
Database connection failed	Kiểm tra DATABASE_URL, MySQL running
Build fails	<code>pnpm clean && pnpm install && pnpm build</code>
tRPC errors	Kiểm tra server logs, validate input types
Slow queries	Thêm indexes, optimize queries

7.2 Debug Mode

```
# Enable debug logging
DEBUG=* pnpm dev

# Check database
mysql -u user -p database
SHOW TABLES;
SELECT COUNT(*) FROM residents;
```

7.3 Performance Tuning

```
# Node.js memory limit
NODE_OPTIONS="--max-old-space-size=2048" node dist/index.js

# Database connection pool
// server/db.ts
const pool = mysql.createPool({
  max: 20,
  min: 5,
  idleTimeoutMillis: 30000,
});
```

8. Maintenance

8.1 Regular Tasks

Task	Tần Suất	Mô Tả
Update dependencies	Hàng tháng	<code>pnpm update</code>
Security patches	Khi có	<code>pnpm audit fix</code>
Database backup	Hàng ngày	Backup database
Log rotation	Hàng tuần	Xóa logs cũ
Performance review	Hàng tháng	Kiểm tra metrics

8.2 Monitoring Checklist

```
# CPU & Memory
top
free -h

# Disk space
df -h

# Network
netstat -an | grep ESTABLISHED | wc -l

# Database
mysql -u user -p -e "SHOW PROCESSLIST;"

# Application logs
tail -f /var/log/residence-care.log
```

8.3 Scaling

Vertical Scaling:

- Tăng CPU, RAM của server

Horizontal Scaling:

- Chạy multiple instances
- Sử dụng load balancer (Nginx, HAProxy)
- Database replication

9. Security Checklist

- ☐ Change default passwords
- ☐ Enable HTTPS/SSL
- ☐ Setup firewall rules
- ☐ Enable database backups
- ☐ Setup monitoring & alerts

- ☐ Regular security updates
 - ☐ Rate limiting enabled
 - ☐ Input validation enabled
 - ☐ CORS properly configured
 - ☐ Secrets not committed to git
-

10. Rollback Procedure

```
# Nếu deployment gặp vấn đề
# Rollback đến checkpoint trước

# Manus Platform: Click "Rollback" button
# Self-hosted: Restore từ backup

# Database rollback
mysql -u user -p database < backup_before_deployment.sql

# Application rollback
git revert <commit-hash>
pnpm build
pm2 restart residence-care
```

Phiên bản tài liệu: 1.0

Cập nhật lần cuối: Tháng 5 năm 2026

Trạng thái: Production Ready